Object Oriented Programming – II

Project Report

Smart Restaurant Order & Analytics System (SROAS)

Student Name: Mukund Kumra

Lecturer Name: Dr. Adrielle Moraes

Project: Smart Restaurant Order & Analytics System (SROAS)

Module: Object Oriented Programming – II

Table of Contents

Introduction 3
Reason for Choosing This Domain 4
User Stories Implemented 5
System Overview & Architecture 7
Class/UML Relationships 9
Data Flow 11
Key Java / OOP Features Used 12
Evaluation & Challenges 13
Version Control 15
Future Improvements 16
Presentation Video Link 17
Conclusion 18

Introduction

The **Smart Restaurant Order & Analytics System (SROAS)** is a Java-based console application written in Java 25 designed to simulate a modern restaurant management system. The application demonstrates both fundamental and advanced OOP concepts required by the Objection Oriented Programming II Project. The application simulates a simplified restaurant ordering workflow, from browsing a menu and placing an order, to kitchen processing and order persistence.

The project is structured into four logical packages: model (data types), service (business logic), util (helpers and localisation), and the root entry point RestaurantApp.java. This layered design keeps concerns separated and makes it straightforward to locate specific Java feature demonstrations.

Reason for Choosing This Domain

I chose this domain (SROAS) as it naturally supports both business logic and advanced Java features. The domain includes:

- **Entities:** Menu items, orders, payments
- **Processes:** Ordering, payment handling, kitchen processing
- **Analytics:** Data aggregation and insights
- **Concurrency:** Multiple orders processed simultaneously

This domain allows meaningful integration of:

- Streams and collectors (analytics)
- Concurrency (kitchen simulation)
- Records (immutable data)
- Sealed classes (controlled payment hierarchy)

User Stories Implemented

To guide the design of the SROAS, a set of user stories was created so that the functionality would reflect realistic user goals rather than just technical features. These stories shaped the structure of the menus and ensured each class existed for a clear purpose.

Fundamental Features

- **US1 - View menu sorted by price**: As a user, I want to see the menu sorted from cheapest to most expensive, so that I can make an informed choice.
- **US2 - Demonstrate lambdas (Consumer, Predicate, Supplier, Function):** As a developer, I want to see all four core functional interfaces used, to confirm understanding of the Lambda feature.
- **US3 - Perform stream operations (min, max, count, etc.):** As a developer, I want every required stream operation demonstrated.
- **US4 - Collect data using grouping and partitioning:** As a developer, I want to see Collectors.toMap(), groupingBy(), and partitioningBy() demonstrated.
- **US5 - Select items and process payment:** As a user, I want to choose items from the menu and specify a payment method, so that an order can be created.
- **US6 - Use sealed payment types:** As a developer, I want to see sealed classes/interfaces preventing arbitrary payment subtypes.
- **US7 - Record order timestamps:** As a user, I want each order to record when it was placed.
- **US8 - Represent data using Records**: As a developer, I want to see Java Records used for immutable data carriers.

Advanced Features

- **US9 - Process orders concurrently:** As a restaurant system, I want orders to be processed concurrently so the kitchen can handle multiple tickets in parallel.
- **US10 - Persist orders using file I/O:** As a system, I want completed orders saved to a file so they are not lost when the application exits.
- **US11 - Support multilingual interface:** As a user, I want to choose between English and Spanish at startup.
- **US12 - Use instance main method (JEP 512):** As a developer, I want to see the Java 25 compact source / instance main method feature.
- **US13 - Flexible constructor bodies (JEP 513):** As a developer, I want to see pre-super/this logic in a constructor.

Extra Features

- **US14 - Scoped Values:** As a developer, I want to see Scoped Values used for safe, structured data propagation.
- **US15 - Stream Gatherers:** As a developer, I want to see the new Stream Gatherers API used.

System Overview & Architecture

Upon starting the application, the user is presented with a textual menu through the console interface. The user can choose to view the menu, select items, and proceed with placing an order by specifying a preferred payment method. Based on the selected options, the system guides the user through the ordering workflow in a structured manner. For example:

- A user can browse menu items and view them sorted by price.
- A user can select multiple items to create an order.
- A user can choose a payment method such as cash, card, or online payment.

Once an order is placed, the system internally creates an Order record containing the selected items, payment type, and timestamp. The order is then passed to the kitchen processing system, where multiple orders can be handled concurrently using a thread pool. After processing, the order is persisted to a file, ensuring that order data is stored beyond the application runtime. In addition, the system performs analytics on menu and order data using Java Streams, enabling operations such as filtering, grouping, and summarising information.

The system is structured into four main layers, separating concerns to improve maintainability and scalability:

1. UI Layer

RestaurantApp handles:

- Console interaction
- User input
- Menu navigation

2. Service Layer

Handles business logic:

- Menu operations
- Order processing
- Analytics
- File handling
- Kitchen concurrency

3. Model Layer

Defines core data structures:

- MenuItem (Record)
- Order (Record)
- Payment (Sealed Interface)

4. Utility Layer

Provides:

- Localisation (LocaleManager)
- Date handling (DateUtil)

Class/UML Relationships

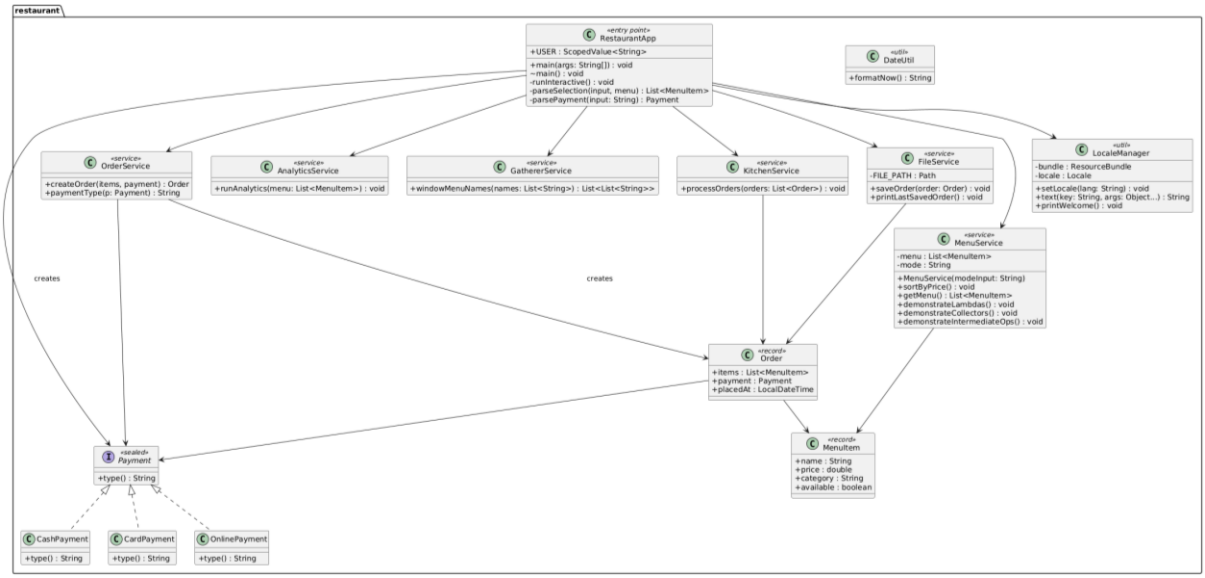

Figure 1: UML Diagram of SROAS

This UML diagram illustrates the overall architecture of the SROAS (Smart Restaurant Ordering and Service) system, showing how its classes, interfaces, enums, and records interact through inheritance, composition, and behavioural contracts. At the core of the domain model is the abstraction of orders and payments, supported by clearly defined relationships between entities, services, and utility components.

On the ordering side, the system revolves around the Order class, which represents a customer's transaction. Each order is composed of multiple MenuItem objects, demonstrating a composition relationship where an order cannot exist without its items. The MenuItem class encapsulates properties such as name, price, and category, and is classified using the Category enum, ensuring that menu items belong to predefined groups such as food or beverages. This use of enums enforces consistency and avoids invalid classifications.

Payment handling is designed using abstraction and polymorphism. The Payment interface defines the common contract for all payment types, such as processing a payment or retrieving payment details. This interface is implemented by multiple concrete classes, including CashPayment and CardPayment, each providing its own specific behaviour. This design allows the system to remain extensible, making it easy to introduce new payment methods in the future without modifying existing logic.

The system's business logic is managed by service classes, primarily the OrderService. This class is responsible for creating and managing orders, calculating totals, and coordinating interactions between menu items and payment types. It acts as the central point where core operations are executed, ensuring separation of concerns between data representation and business rules.

To support unique identification and tracking, the system uses universally unique identifiers (UUIDs) for orders, ensuring that each transaction can be distinctly recognised. Additionally, time-related operations, such as recording when an order is created, are handled using date-time utilities, reinforcing accuracy and traceability.

The application flow is orchestrated by the main entry point class, typically RestaurantApp. This class acts as the interface between the user and the system, handling input, triggering service-layer operations, and displaying outputs. It connects all components together, ensuring that user actions such as placing an order or making a payment are correctly processed through the underlying architecture.

Overall, the diagram demonstrates a clean separation between interfaces, implementations, and services. It highlights the use of object-oriented principles such as encapsulation, inheritance, and polymorphism to create a modular, extensible, and maintainable system design.

Data Flow

System Flow:

1. Start Application
2. User selects language
3. Menu is displayed
4. Features are demonstrated (Lambdas, Collectors, Analytics, Gatherers)
5. User selects items from Menu
6. Payment method is chosen
7. Order is created
8. Kitchen processes order (concurrently)
9. Order is saved to file (orders.txt)

This flow mirrors real-world restaurant operations and demonstrates structured data movement.

Key Java / OOP Features Used

Fundamentals

- Encapsulation through service classes
- Records for immutable data
- Functional interfaces (Consumer, Predicate, Supplier, Function)
- Streams (filter, map, sorted, etc.)
- Switch expressions with pattern matching

Advanced Features

- **Concurrency:** ExecutorService + Callable
- **File Handling:** NIO2 (Files, Path, StandardOpenOption)
- **Localisation:** ResourceBundle
- **Sealed Classes:** Controlled inheritance
- **Date/Time API:** LocalDateTime, ZonedDateTime

Modern Java (Java 25)

- Instance main method (JEP 512)
- Flexible constructors (JEP 513)
- Scoped Values
- Stream Gatherers

Evaluation & Challenges

Developing the SROAS system was a valuable learning experience that allowed me to apply modern Java concepts in a practical, real-world style application. The project successfully meets the assignment requirements while incorporating both foundational and advanced features, though several challenges were encountered throughout the development process.

One of the primary challenges involved working with Java 25 preview features. Ensuring that newer constructs such as enhanced stream capabilities and modern language features were correctly configured and supported within the development environment required careful setup and troubleshooting. This process improved my understanding of how evolving Java features integrate with existing tooling.

Another challenge was ensuring that a wide range of stream operations were meaningfully demonstrated. While streams provide a concise and expressive way to process data, designing scenarios where operations such as filtering, mapping, and sorting were both necessary and logically integrated required thoughtful planning. This strengthened my ability to apply functional programming techniques in a purposeful way rather than using them superficially.

Concurrency also presented a significant challenge. Implementing asynchronous operations using ExecutorService and Callable required careful consideration of thread safety and task management. Avoiding issues such as race conditions and ensuring predictable execution helped build a stronger understanding of concurrent programming principles.

Structuring the codebase for clarity and assessment was another important consideration. Balancing clean architectural design with the need to explicitly demonstrate required features meant that decisions around class organisation, service layers, and separation of concerns had to be made carefully.

Overall, this project significantly strengthened my understanding of modern Java. I gained practical experience in applying functional programming concepts, improved my ability to work with concurrency, and developed better architectural design skills. The experience has enhanced my confidence in building scalable, maintainable systems using up-to-date Java practices.

Strengths and Weaknesses

Strengths

- Complete feature coverage
- Clean layered architecture
- Use of modern Java features
- Real-world simulation
- Localisation support

Challenges

- **Console-based UI only**
  Limited user interaction compared to graphical interfaces.
- **Limited validation for user input**
  Basic validation is implemented but could be expanded for robustness.
- **Integration of multiple advanced features**
  Combining concurrency, streams, localisation, and modern Java features increased overall system complexity.
- **Maintaining readability with functional programming**
  Extensive use of streams and lambdas can reduce readability if not carefully structured.
- **Debugging concurrent operations**
  Identifying issues in multithreaded execution (e.g., task ordering or unexpected behaviour) was more difficult than in single-threaded code.

Version Control

The project was tracked using Git and kept in a public GitHub repository to facilitate development and keep an accurate record of progress. I was able to maintain a clear history of modifications, create backups of the codebase, and make frequent contributions as features were added by using Git. The repository shows regular updates throughout development and includes the complete source code, diagrams, and project report. This directly satisfies the assignment's requirements for additional marks for using a public repository. The repository's link is:

https://github.com/mukundkumra/sroas

Future Improvements

Although the SROAS satisfies project requirements, it can be improved in numerous ways in the future. Some possible improvements may look like:

- GUI (JavaFX or Web UI)
- REST API integration
- Database storage (MySQL/PostgreSQL)
- Improved validation and error handling
- Unit testing (JUnit)
- Enhanced analytics dashboard

These additions would further extend the system and make it more realistic for real-world deployment.

Presentation Video Link

The video presentation for the project can be viewed here:

Java Project Presentation Video

Conclusion

In conclusion, the Smart Restaurant Order & Analytics System effectively demonstrates the full range of object-oriented and advanced Java features required for this module. The system integrates object-oriented design principles with modern programming practices, including functional programming, concurrency, file handling, internationalisation, and the use of recent Java 25 features.

Throughout development, challenges related to integrating advanced features, managing concurrency, and structuring the system for clarity contributed to a deeper understanding of modern Java. These challenges helped improve both the design and overall robustness of the application.

The project meets the assignment requirements, provides a strong architectural foundation, and demonstrates a high level of technical competence. It also leaves scope for future enhancements such as implementing persistent storage, improving input validation, and developing a graphical user interface, making it a scalable and extensible system.